

Lab 14 - Namespaces, ServiceAccounts and RBAC

Field	Value
Lab ID	Lab 14
Day / Topic	Day 3 Security
Duration	55 minutes
Namespace	kcna-lab14
Mapping	LO4 A4 K6
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

1. Objective

By the end of this lab you will be able to:

1. State precisely what a Namespace does and does not isolate.
2. Create ServiceAccounts and explain that an unbound ServiceAccount has zero permissions.
3. Read the projected ServiceAccount token volume that kubelet mounts into a Pod, and disable it with automountServiceAccountToken: false.
4. Distinguish Role vs ClusterRole and RoleBinding vs ClusterRoleBinding, and explain the crucial third combination: a *RoleBinding* pointing at a *ClusterRole*.
5. Use `kubectl auth can-i --as=system:serviceaccount:<ns>:<sa>` as the authoritative proof tool, and drive a whole permission matrix through it.
6. Diagnose the real Forbidden error, the silent dangling roleRef, and the immutable roleRef rejection.

2. Prerequisites

- A running single-node kind cluster where your kubeconfig user is cluster-admin

(the kind default). You need the impersonate permission for `--as`.

- Working directory:

```
cd courseware/labs/lab-14-namespaces-serviceaccounts-rbac
pwd
```

Expected output:

```
/.../courseware/labs/lab-14-namespaces-serviceaccounts-rbac
```

- Confirm RBAC is the active authorization mode. If it is not, nothing in this lab

will deny anything:

```
kubectl api-versions | grep rbac
```

Expected output:

```
rbac.authorization.k8s.io/v1
```

```
kubectl auth can-i create clusterrolebindings
```

Expected output (you are cluster-admin on kind):

```
yes
```

3. Scenario

Meridian Freight has one shared cluster. The night-shift operations team must be able to look at the freight platform's Pods and read their logs when a booking job fails at 03:00 - and must not be able to change anything, read any credential, or see into any other team's namespace.

Separately, a CI identity called deploy-bot has been created in advance for a pipeline that is not built yet. It must have no access until someone deliberately grants it.

You will implement both, and prove each claim with a permission matrix rather than an opinion.

4. Step-by-step procedure

Step 1 - Create the namespace and read what it actually gives you

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab14 created
```

```
kubectl get namespace kcna-lab14 --show-labels
```

Expected output:

NAME	STATUS	AGE	LABELS
kcna-lab14	Active	6s	app.kubernetes.io/part-of=meridian-freight, kcna.tertiaryinfotech.com/day=3, kcna.tertiaryinfotech.com/lab=lab-14, kubernetes.io/metadata.name=kcna-lab14, pod-security.kubernetes.io/enforce=baseline, pod-security.kubernetes.io/warn=restricted

Note kubernetes.io/metadata.name=kcna-lab14 - the API server adds this label to every Namespace automatically. Lab 16 depends on it, because a NetworkPolicy namespaceSelector can only select on labels.

What a Namespace is and is not:

A Namespace gives you	A Namespace does not give you
A unique name scope for namespaced objects	Network isolation - by default any Pod can reach any Pod in any namespace (Lab 16)
A target for RoleBinding - the unit of RBAC scoping	Protection from any subject holding a ClusterRoleBinding
A target for ResourceQuota and LimitRange	Node, kernel, CPU or memory isolation on its own
A target for Pod Security Admission labels	Isolation of cluster-scoped objects: Nodes, PersistentVolumes, StorageClasses, ClusterRoles
A deletion unit - delete the namespace, delete its contents	Any guarantee about what a workload's *image* does

Confirm the last row of the left column - the cluster-scoped kinds that a Namespace can never contain:

```
kubectl api-resources --namespaced=false --api-group='' -o name
```

Expected output:

```
componentstatuses
namespaces
nodes
persistentvolumes
```

Step 2 - ServiceAccounts, and the fact that they grant nothing

```
kubectl apply -f manifests/10-serviceaccounts.yaml
```

Expected output:

```
serviceaccount/ops-reader created
serviceaccount/deploy-bot created
```

```
kubectl get serviceaccounts -n kcna-lab14
```

Expected output - note the third one you did not create:

```
NAME          SECRETS  AGE
default       0        55s
deploy-bot    0        4s
ops-reader    0        4s
```

Two observations:

- default was created for you. Every namespace gets one, and every Pod that does

not name a ServiceAccount runs as it.

- SECRETS is 0. Before Kubernetes v1.24 each ServiceAccount had an auto-created,

never-expiring Secret token. That behaviour was removed; tokens are now short-lived and projected into Pods, or minted on demand with `kubectl create token`.

Now prove the important point - a brand-new ServiceAccount can do nothing:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
```

Expected output (exit code 1):

```
no
```

The identity exists. It has no permissions. Creating a ServiceAccount is not granting access.

Note the subject string format, which you will type all lab: system:serviceaccount:<namespace>:<name>. Every ServiceAccount is also a member of the group system:serviceaccounts:<namespace>.

Step 3 - Grant read-only Pod access with a namespaced Role

```
kubectl apply -f manifests/20-role-pod-reader.yaml
```

Expected output:

```
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/ops-reader-can-read-pods created
```

```
kubectl describe role pod-reader -n kcna-lab14
```

Expected output:

```
Name:          pod-reader
Labels:        app.kubernetes.io/part-of=meridian-freight
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  pods      []                  []              [get list watch]
  pods/log   []                  []              [get]
```

Read the rule model out loud: apiGroups × resources × verbs, optionally narrowed by resourceNames. RBAC is purely additive - there is no deny rule. A subject can do something if *any* bound rule allows it, and nothing else can take that away.

Note that pods/log needed its own rule. Granting pods does not grant pods/log, pods/exec or pods/portforward. pods/exec is effectively shell access to the container - never grant it casually.

Now re-run the same question as Step 2:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
```

Expected output:

```
yes
```

And the scoping claim - the same subject, a different namespace:

```
kubectl auth can-i list pods --as=system:serviceaccount:kcna-lab14:ops-reader -n default
```

Expected output:

```
no
```

The RoleBinding, not the Role, decides the namespace.

Step 4 - A ClusterRole scoped by a RoleBinding

Meridian Freight wants the same "view config" rule set in many namespaces without copying a Role into each. That is what a ClusterRole is for.

```
kubectl apply -f manifests/30-clusterrole-configmap-viewer.yaml
```

Expected output:

```
clusterrole.rbac.authorization.k8s.io/kcna-lab14-configmap-viewer created
rolebinding.rbac.authorization.k8s.io/ops-reader-can-view-configmaps created
```

This ClusterRole is the only cluster-scoped object this lab creates. It is

read-only (get, list, watch on configmaps), has no wildcards, and - this is

the part that matters - is bound by a RoleBinding, not a ClusterRoleBinding. No

privilege is widened beyond kcna-lab14. Section 8 deletes it by exact name.

```
kubectl describe clusterrole kcna-lab14-configmap-viewer
```

Expected output:

```
Name:          kcna-lab14-configmap-viewer
Labels:        app.kubernetes.io/part-of=meridian-freight
               kcna.tertiaryinfotech.com/lab=lab-14
Annotations:   <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  -----  -
  configmaps  []                      []              [get list watch]
```

The four combinations, which are examinable:

Role kind	Binding kind	Where the permissions apply
Role	RoleBinding	The RoleBinding's namespace only
ClusterRole	RoleBinding	The RoleBinding's namespace only - the rule set is reusable, the grant is not cluster-wide
ClusterRole	ClusterRoleBinding	Every namespace, plus cluster-scoped resources
Role	ClusterRoleBinding	Illegal. A ClusterRoleBinding's roleRef.kind must be ClusterRole

Prove row 2 - the grant did not leak out of the namespace:

```
kubectl auth can-i list configmaps --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
kubectl auth can-i list configmaps --as=system:serviceaccount:kcna-lab14:ops-reader -n default
```

Expected output:

```
yes
no
```

Confirm no ClusterRoleBinding was created anywhere that references it:

```
kubectl get clusterrolebindings -o jsonpath='{range .items[?(@.roleRef.name=="kcna-lab14-configmap-viewer")]}{.metadata.name}{"\n"}{end}'
echo "(empty above = no cluster-wide grant exists)"
```

Expected output:

```
(empty above = no cluster-wide grant exists)
```

For contrast, inspect a built-in read-only ClusterRole that ships with every cluster - do not bind it, just read it:

```
kubectl describe clusterrole view | head -14
```

Expected output (abridged; the full list is long):

```
Name:      view
Labels:    kubernetes.io/bootstrapping=rbac-defaults
Annotations: rbac.authorization.k8s.io/aggregate-to-edit=true
            rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
  Resources            Non-Resource URLs  Resource Names  Verbs
  -----
  bindings              []                  []              [get list
watch]
  configmaps            []                  []              [get list
watch]
  endpoints             []                  []              [get list
watch]
  events                []                  []              [get list
watch]
  limitranges           []                  []              [get list
watch]
  namespaces/status     []                  []              [get list
watch]
  namespaces            []                  []              [get list
watch]
  persistentvolumeclaims/status
watch]                []                  []              [get list
```

view deliberately excludes Secrets. edit adds write verbs. admin adds RBAC management within a namespace. cluster-admin is * on * - bound to the system:masters group, which is what your kubeconfig uses on kind.

Step 5 - Drive the whole permission matrix from data/

```
cat data/rbac-matrix.csv
```

Expected output:

```

serviceaccount,verb,resource,namespace,expected,granted_by
ops-reader,get,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,list,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,watch,pods,kcna-lab14,yes,Role/pod-reader via RoleBinding
ops-reader,get,pods/log,kcna-lab14,yes,Role/pod-reader subresource rule
ops-reader,list,configmaps,kcna-lab14,yes,ClusterRole/kcna-lab14-configmap-viewer scoped by a
RoleBinding
ops-reader,delete,pods,kcna-lab14,no,verb not present in any bound rule
ops-reader,list,secrets,kcna-lab14,no,resource not present in any bound rule
ops-reader,list,pods,default,no,RoleBinding only grants inside kcna-lab14
ops-reader,list,configmaps,default,no,a RoleBinding to a ClusterRole still only grants in its
own namespace
deploy-bot,list,pods,kcna-lab14,no,ServiceAccount exists but has no binding at all
deploy-bot,create,deployments,kcna-lab14,no,ServiceAccount exists but has no binding at all
default,list,pods,kcna-lab14,no,the auto-created default ServiceAccount is unprivileged

```

Spot-check three rows by hand before letting the script do all twelve:

```

kubectl auth can-i get pods/log --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
kubectl auth can-i delete pods --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14
kubectl auth can-i create deployments --as=system:serviceaccount:kcna-lab14:deploy-bot -n kcna-
lab14

```

Expected output:

```

yes
no
no

```

Ask the API server to enumerate everything the subject can do - the fastest way to audit an identity:

```

kubectl auth can-i --list --as=system:serviceaccount:kcna-lab14:ops-reader -n kcna-lab14

```

Expected output (row order varies; the last block is granted to every authenticated identity by the built-in system:discovery and system:basic-user ClusterRoles):

Resources	Non-Resource URLs	Resource Names	Verbs
selfsubjectreviews.authentication.k8s.io	[]	[]	[create]
selfsubjectaccessreviews.authorization.k8s.io	[]	[]	[create]
selfsubjectrulesreviews.authorization.k8s.io	[]	[]	[create]
configmaps	[]	[]	[get list
watch]			
pods	[]	[]	[get list
watch]			
pods/log	[]	[]	[get]
	[/api/*]	[]	[get]
	[/api]	[]	[get]
	[/healthz]	[]	[get]
	[/livez]	[]	[get]
	[/openapi/*]	[]	[get]
	[/openapi]	[]	[get]
	[/readyz]	[]	[get]
	[/version/]	[]	[get]
	[/version]	[]	[get]

Exactly the three resource lines you granted, plus the harmless discovery URLs. No secrets. No write verbs.

Step 6 - The projected ServiceAccount token inside a Pod

```
kubectl apply -f manifests/40-pod-token-inspector.yaml
kubectl apply -f manifests/50-pod-no-token.yaml
kubectl wait --for=condition=Ready pod/token-inspector pod/no-token -n kcna-lab14 --timeout=90s
```

Expected output:

```
pod/token-inspector created
pod/no-token created
pod/token-inspector condition met
pod/no-token condition met
```

Look at what kubelet mounted:

```
kubectl exec -n kcna-lab14 token-inspector -- ls -l /var/run/secrets/kubernetes.io/
serviceaccount/
```

Expected output (timestamps differ):

```
total 0
lrwxrwxrwx    1 root    root          13 Sep  5 10:02 ca.crt -> ../data/ca.crt
lrwxrwxrwx    1 root    root          16 Sep  5 10:02 namespace -> ../data/namespace
lrwxrwxrwx    1 root    root          12 Sep  5 10:02 token -> ../data/token
```

Three files, and the symlink-to-../data pattern that lets kubelet swap the contents atomically when the token is rotated.

```
kubectl exec -n kcna-lab14 token-inspector -- cat /var/run/secrets/kubernetes.io/
serviceaccount/namespace
```

Expected output (no trailing newline):

```
kcna-lab14
```

```
kubectl exec -n kcna-lab14 token-inspector -- sh -c 'wc -c < /var/run/secrets/kubernetes.io/
serviceaccount/token'
```

Expected output (a JWT; the exact length varies):

```
1012
```

Now the part learners rarely see - where that mount came from. You never wrote a volume:

```
kubectl get pod token-inspector -n kcna-lab14 -o jsonpath='{.spec.volumes[0]}' ; echo
```

Expected output (the volume name suffix is random):

```
{
  "name": "kube-api-access-7v2qd",
  "projected": {
    "defaultMode": 420,
    "sources": [
      {
        "serviceAccountToken": {
          "expirationSeconds": 3607,
          "path": "token"
        },
        "configMap": {
          "items": [
            {
              "key": "ca.crt",
              "path": "ca.crt"
            }
          ],
          "name": "kube-root-ca.crt"
        },
        "downwardAPI": {
          "items": [
            {
              "fieldRef": {
                "apiVersion": "v1",
                "fieldPath": "metadata.namespace"
              },
              "path": "namespace"
            }
          ]
        }
      }
    ]
  }
}
```

Read the three sources:

Source	Produces	Note
serviceAccountToken	token	expirationSeconds: 3607 - roughly one hour. kubelet rotates it and rewrites the file. The token is also audience-bound and bound to this Pod's UID, so it stops working the moment the Pod is deleted
configMap: kube-root-ca.crt	ca.crt	The cluster CA, so the workload can verify the API server's certificate
downwardAPI	namespace	The Pod's own namespace

This is `*BoundServiceAccountTokenVolume*`: short-lived, audience- and object-bound credentials replacing the old forever-valid Secret tokens.

Now the contrast:

```
kubectl exec -n kcna-lab14 no-token -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Expected output (exit code 1 - the directory does not exist):

```
ls: /var/run/secrets/kubernetes.io/serviceaccount/: No such file or directory
command terminated with exit code 1
```

```
kubectl get pod no-token -n kcna-lab14 -o jsonpath='{.spec.volumes}' ; echo
```

Expected output:

No volumes at all. `automountServiceAccountToken: false` is free defence in depth for every workload that does not call the Kubernetes API - which is most of them. You can also set it on the ServiceAccount to make it the default for all Pods using it.

Finally, mint a token by hand - the supported replacement for the removed auto-generated Secret:

```
kubectl create token ops-reader -n kcna-lab14 --duration=10m | cut -c1-45
```

Expected output (a different token every time):

```
eyJhbGciOiJSUzI1NiIsImtpZCI6IlFrTndFcXBGYlZo
```

A JWT is three base64url segments separated by dots - header, payload, signature. The payload carries `aud`, `exp`, and a `kubernetes.io` claim naming the namespace, ServiceAccount and (for projected tokens) the Pod. It is signed, not encrypted - anyone holding it can read those claims, and anyone holding it can `*use*` it. Treat a token like a password.

5. Verification

```
bash verification/checks.sh
```

Expected output:

```

== Lab 14 verification: Namespaces, ServiceAccounts and RBAC ==
PASS namespace kcna-lab14 exists
PASS namespace enforces the baseline Pod Security profile
PASS ServiceAccounts ops-reader and deploy-bot exist
PASS Role pod-reader is read-only (no create/update/patch/delete verbs)
PASS RoleBinding ops-reader-can-read-pods binds Role/pod-reader to ops-reader
PASS ClusterRole kcna-lab14-configmap-viewer is read-only with no wildcards
PASS NO ClusterRoleBinding references kcna-lab14-configmap-viewer (grant stays namespaced)
PASS can-i get pods as ops-reader in kcna-lab14 = yes
PASS can-i list pods as ops-reader in kcna-lab14 = yes
PASS can-i watch pods as ops-reader in kcna-lab14 = yes
PASS can-i get pods/log as ops-reader in kcna-lab14 = yes
PASS can-i list configmaps as ops-reader in kcna-lab14 = yes
PASS can-i delete pods as ops-reader in kcna-lab14 = no
PASS can-i list secrets as ops-reader in kcna-lab14 = no
PASS can-i list pods as ops-reader in default = no
PASS can-i list configmaps as ops-reader in default = no
PASS can-i list pods as deploy-bot in kcna-lab14 = no
PASS can-i create deployments as deploy-bot in kcna-lab14 = no
PASS can-i list pods as default in kcna-lab14 = no
PASS token-inspector has a projected serviceAccountToken volume
PASS token-inspector namespace file contains kcna-lab14
PASS no-token Pod has no ServiceAccount volume mounted
-- 22 passed, 0 failed --

```

6. Failure injection

6.1 The real Forbidden - a subject missing a verb or a resource

```
kubectl get secrets -n kcna-lab14 --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output (exit code 1):

```
Error from server (Forbidden): secrets is forbidden: User "system:serviceaccount:kcna-lab14:ops-reader" cannot list resource "secrets" in API group "" in the namespace "kcna-lab14"
```

Parse the message field by field - it tells you everything you need:

Fragment	Meaning
User "system:serviceaccount:kcna-lab14:ops-reader"	The subject the request was evaluated as
cannot list	The verb. list and get are different verbs; granting get alone does not allow kubectl get pods without a name
resource "secrets"	The resource, plural, as it appears in a rule
in API group ""	The apiGroup. "" is the core group. A Deployment would say in API group "apps"
in the namespace "kcna-lab14"	The scope. If a request against a cluster-scoped resource fails, this clause is absent

Those five fields map exactly onto the five fields of an RBAC rule. Now the same subject, same verb, same resource, different namespace:

```
kubectl get pods -n default --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output:

```
Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kcna-lab14:ops-reader" cannot list resource "pods" in API group "" in the namespace "default"
```

Compare with the same command inside the granted namespace:

```
kubectl get pods -n kcna-lab14 --as=system:serviceaccount:kcna-lab14:ops-reader
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
no-token	1/1	Running	0	3m
token-inspector	1/1	Running	0	3m

6.2 The silent failure - a dangling roleRef

```
kubectl apply -f manifests/60-rolebinding-dangling.yaml
```

Expected output - created without complaint:

```
rolebinding.rbac.authorization.k8s.io/deploy-bot-dangling created
```

```
kubectl auth can-i create deployments --as=system:serviceaccount:kcna-lab14:deploy-bot -n kcna-lab14
```

Expected output:

```
no
```

The RoleBinding exists. The permission does not. Diagnose it:

```
kubectl describe rolebinding deploy-bot-dangling -n kcna-lab14
```

Expected output:

```
Name:          deploy-bot-dangling
Labels:        kcna.tertiaryinfotech.com/intent=failure-injection
Annotations:   <none>
Role:
  Kind:  Role
  Name:  deployment-manager
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount  deploy-bot  kcna-lab14
```

Nothing here says the Role is missing. Ask directly:

```
kubectl get role deployment-manager -n kcna-lab14
```

Expected output:

```
Error from server (NotFound): roles.rbac.authorization.k8s.io "deployment-manager" not found
```

Diagnosis: roleRef is not validated on write. RBAC treats a binding to a non-existent Role as granting nothing, and emits no error, warning or event. The triage routine for "I made a RoleBinding and it still says Forbidden":

```
kubectl get rolebindings -n kcna-lab14 -o custom-columns='NAME:.metadata.name,ROLEKIND:.roleRef.kind,ROLE:.roleRef.name,SUBJECT:.subjects[0].name'
```

Expected output:

NAME	ROLEKIND	ROLE	SUBJECT
deploy-bot-dangling	Role	deployment-manager	deploy-bot
ops-reader-can-read-pods	Role	pod-reader	ops-reader
ops-reader-can-view-configmaps	ClusterRole	kcna-lab14-configmap-viewer	ops-reader

Then confirm each ROLE actually exists:

```
kubectl get roles -n kcna-lab14 -o name
```

Expected output:

```
role.rbac.authorization.k8s.io/pod-reader
```

deployment-manager is absent - that is the fault. The three other classic causes of the same symptom, in the order worth checking:

1. subjects[].namespace omitted for a ServiceAccount subject (it is required; it does not default to the RoleBinding's namespace).
2. subjects[].name misspelled - RBAC does not verify subjects exist either.
3. The RoleBinding is in the wrong namespace.

Remove the broken binding:

```
kubectl delete rolebinding deploy-bot-dangling -n kcna-lab14
```

Expected output:

```
rolebinding.rbac.authorization.k8s.io "deploy-bot-dangling" deleted
```

6.3 The immutable roleRef

Try to repoint an existing binding at a different Role:

```
kubectl patch rolebinding ops-reader-can-read-pods -n kcna-lab14 --type=merge \
  -p '{"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"Role","name":"pod-reader-v2"}}'
```

Expected output (rejected):

```
The RoleBinding "ops-reader-can-read-pods" is invalid: roleRef: Invalid value: rbac.RoleRef{APIGroup:"rbac.authorization.k8s.io", Kind:"Role", Name:"pod-reader-v2"}: cannot change roleRef
```

Diagnosis: roleRef is immutable by design. Silently repointing a binding is a privilege-escalation vector, so the API server forces you to delete and recreate, which is auditable. subjects can be edited freely.

Confirm the original binding is untouched:

```
kubectl get rolebinding ops-reader-can-read-pods -n kcna-lab14 -o jsonpath='{.roleRef.name}'\n\"}
```

Expected output:

```
pod-reader
```

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix
Error from server (Forbidden): ... cannot list resource "secrets" in API group ""	No bound rule covers that verb+resource+group	kubectl auth can-i --list --as=<subject> -n <ns>	Add the resource/verb to a Role, or bind an existing one. Never reach for cluster-admin
RoleBinding created but access still denied, no error anywhere	roleRef names a Role that does not exist	kubectl get role <roleRef.name> -n <ns> → NotFound	Create the Role, or delete and recreate the binding pointing at the right one
Same, but the Role exists	subjects[].namespace missing on a ServiceAccount subject, or the name is misspelled	kubectl get rolebinding <n> -o jsonpath='{.subjects}'	Add namespace:: RBAC does not validate subjects
cannot change roleRef	roleRef is immutable	Read the error	kubectl delete rolebinding <n> -n <ns> then recreate
kubectl get pods fails but kubectl get pod <name> works	list and get are separate verbs	kubectl auth can-i list pods --as=<subject> → no	Add list (and watch for -w and for informers)
kubectl logs fails though pods is granted	pods/log is a separate subresource	kubectl auth can-i get pods/log --as=<subject>	Add a rule for pods/log
Deployment access denied though the rule says deployments	Wrong apiGroups - Deployments are in apps, not ""	The Forbidden message prints in API group "apps"	Use apiGroups: ["apps"]
error: You must be logged in to the server from --as=...	Your own user lacks impersonate	kubectl auth can-i impersonate serviceaccounts	Run as cluster-admin, or test from inside a Pod instead
Permissions appear cluster-wide when you wanted them namespaced	A ClusterRoleBinding was used instead of a RoleBinding	kubectl get clusterrolebindings -o jsonpath='{...roleRef.name...}'	Replace with a RoleBinding in the target namespace

8. Cleanup

```
kubectl delete namespace kcna-lab14
```

Expected output:

```
namespace "kcna-lab14" deleted
```

That removes the ServiceAccounts, Role, both RoleBindings and both Pods.

One cluster-scoped object remains. kcna-lab14-configmap-viewer is a

ClusterRole and therefore lives outside every namespace, so deleting the namespace

cannot remove it. Delete it by exact name - never with a selector or --all:

```
kubectl delete clusterrole kcna-lab14-configmap-viewer
```

Expected output:

```
clusterrole.rbac.authorization.k8s.io "kcna-lab14-configmap-viewer" deleted
```

Confirm nothing of this lab is left cluster-wide:

```
kubectl get clusterrole kcna-lab14-configmap-viewer
kubectl get namespace kcna-lab14
```

Expected output:

```
Error from server (NotFound): clusterroles.rbac.authorization.k8s.io "kcna-lab14-configmap-viewer" not found
Error from server (NotFound): namespaces "kcna-lab14" not found
```

Never run `kubectl delete clusterrole --all` or `kubectl delete rolebinding --all`.

The first would destroy the cluster's built-in RBAC and the cluster with it.

9. What you learned

- A Namespace is a scope, not a security boundary. It scopes names, RBAC bindings,

quotas and Pod Security labels - it does not isolate the network, the node, or a subject holding a ClusterRoleBinding.

- A ServiceAccount is an identity with no permissions. Creating one grants nothing.
- Permissions come from a binding. Role+RoleBinding is namespaced;

ClusterRole+RoleBinding is a reusable rule set applied within one namespace; ClusterRole+ClusterRoleBinding is cluster-wide and is what you avoid by default.

- RBAC is additive only; there is no deny rule and no ordering.
- Subresources (pods/log, pods/exec) need their own rules, and the apiGroup

matters ("" for Pods, apps for Deployments).

- kubelet projects a short-lived, audience- and Pod-bound token into every Pod

unless you set `automountServiceAccountToken: false` - which you should, by default.

- `kubectl auth can-i --as=system:serviceaccount:<ns>:<sa>` is the proof tool, and

--list audits an identity in one command.

- The two silent RBAC failures - a dangling roleRef and a missing

subjects[].namespace - produce no error at all. The immutable roleRef is the one that **does** shout, and it does so to prevent silent escalation.

10. Further reading

- Using RBAC Authorization - <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- ServiceAccounts - <https://kubernetes.io/docs/concepts/security/service-accounts/>
- Configure Service Accounts for Pods - <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>
- Namespaces - <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/>
- Authorization overview - <https://kubernetes.io/docs/reference/access-authn-authz/authorization/>
- Pod Security Admission - <https://kubernetes.io/docs/concepts/security/pod-security-admission/>
- Kubernetes API access-control good practices - <https://kubernetes.io/docs/concepts/security/rbac-good-practices/>